

Lesson 1: Introduction to RAG

What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is a technique that combines the power of large language models (LLMs) with external knowledge retrieval. Think of it as giving an AI assistant a library of reference books it can consult before answering your questions.

The Problem RAG Solves

Without RAG: The Limitations of Standard LLMs

Large language models like GPT-4, Claude, or Llama are trained on massive datasets, but they have fundamental limitations:

1. **Knowledge Cutoff:** They only know information from their training data
2. GPT-4 might not know about events after its training cutoff
3. They can't access your company's internal documents

They don't know about your personal files or preferences

Hallucinations: They sometimes make up convincing-sounding but false information

6. When unsure, they might fabricate facts rather than admit uncertainty

No way to verify where information comes from

No Source Attribution: They can't cite their sources

9. Hard to verify accuracy

Can't trace back to original documents

Static Knowledge: They can't be updated without retraining

12. Expensive and time-consuming to update

13. Can't easily add domain-specific knowledge

Example: The Problem in Action

Without RAG:

User: "What was discussed in last week's company meeting?"

LLM: "I don't have access to your company's meetings. I can only provide general information"

With RAG:

User: "What was discussed in last week's company meeting?"

System: [Retrieves meeting notes from company database]

LLM: "Based on the meeting notes from December 27, 2025, the main topics were:

1. Q4 revenue targets exceeded by 15%
2. New product launch scheduled for February
3. ..."

How RAG Works: The Three-Step Process

RAG systems operate in three main steps:

Step 1: Indexing (Done Once)

Before the system can retrieve information, you need to prepare your knowledge base:

1. Collect documents (PDFs, web pages, databases, etc.)
2. Split documents into chunks (paragraphs or sections)
3. Convert chunks into vector embeddings (numerical representations)
4. Store embeddings in a vector database

Example:

```
# Document
"The Eiffel Tower is 330 meters tall and located in Paris, France."

# Split into chunk (in this case, one chunk)
chunk = "The Eiffel Tower is 330 meters tall and located in Paris, France."

# Convert to embedding (simplified)
embedding = [0.23, -0.45, 0.67, ..., 0.12] # 1536 dimensions typically

# Store in database
vector_db.add(chunk, embedding)
```

Step 2: Retrieval (Happens Per Query)

When a user asks a question:

1. Convert the question into an embedding (same process as above)
2. Search the vector database for similar embeddings
3. Retrieve the most relevant document chunks

Example:

```
# User query
query = "How tall is the Eiffel Tower?"

# Convert to embedding
query_embedding = [0.25, -0.43, 0.65, ..., 0.15] # Similar to document

# Find most similar chunks
similar_chunks = vector_db.search(query_embedding, top_k=3)
# Returns: ["The Eiffel Tower is 330 meters tall...", ...]
```

Step 3: Generation (Happens Per Query)

The LLM generates a response using the retrieved context:

1. Create a prompt with the retrieved chunks and the user's question
2. Send to the LLM
3. LLM generates an answer grounded in the retrieved information

Example:

```
# Create augmented prompt
prompt = f"""
Context: {similar_chunks}

Question: {query}

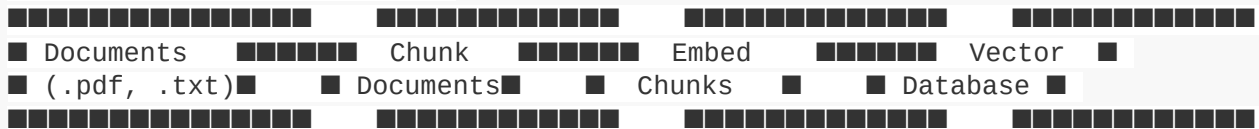
Answer based on the context above:
"""

# Generate response
response = llm.generate(prompt)
# Returns: "The Eiffel Tower is 330 meters tall."
```

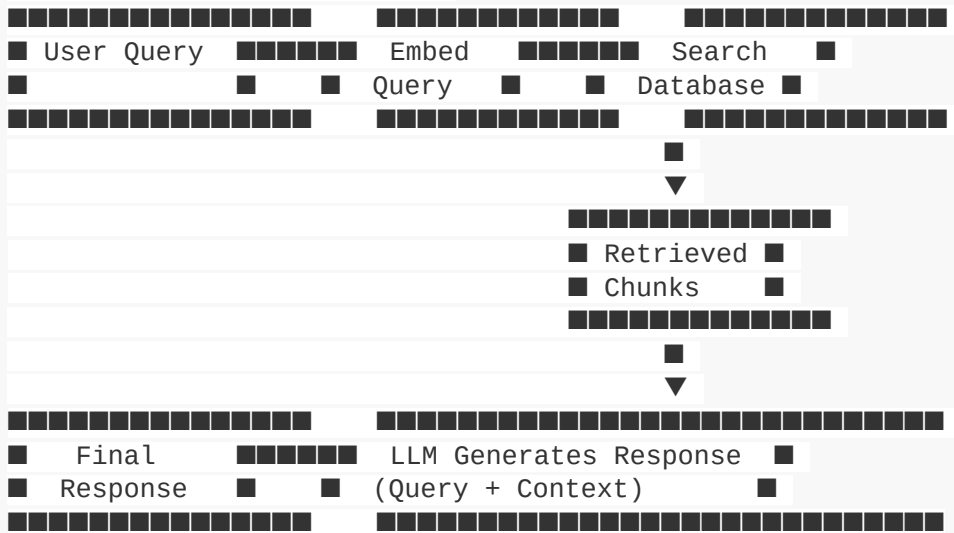
Visual Overview



INDEXING PHASE (Done Once):



QUERY PHASE (Per Question):



Real-World Applications

1. Customer Support Chatbots

- **Knowledge Base:** Product manuals, FAQs, troubleshooting guides
- **Benefit:** Accurate answers grounded in official documentation
- **Example:** "How do I reset my password?" → Retrieves exact steps from manual

2. Research Assistants

- **Knowledge Base:** Scientific papers, research notes, articles

- **Benefit:** Find relevant information across thousands of documents
- **Example:** "What studies have shown a link between sleep and memory?" → Retrieves relevant papers

3. Enterprise Search

- **Knowledge Base:** Internal wikis, documents, emails, databases
- **Benefit:** Natural language interface to company knowledge
- **Example:** "What's our vacation policy?" → Retrieves from HR documentation

4. Personal Knowledge Management

- **Knowledge Base:** Your notes, documents, bookmarks
- **Benefit:** Conversational access to your information
- **Example:** "What were my key takeaways from that ML course?" → Retrieves your notes

5. Legal/Medical AI

- **Knowledge Base:** Case law, regulations, medical literature

- **Benefit:** Grounded responses with source attribution
- **Example:** "What precedents exist for this type of case?" → Retrieves relevant cases

Key Advantages of RAG

1. ■ Up-to-date Information

2. Update knowledge base without retraining the model

Add new documents instantly

■ Domain-Specific Knowledge

5. Works with specialized, proprietary, or personal information

No need to fine-tune expensive models

■ Source Attribution

8. Can cite which documents were used

Users can verify information

■ Reduced Hallucinations

11. Responses grounded in actual documents

Clear when information isn't available

■ Cost-Effective

14. No model retraining required

Works with existing LLMs

■ Privacy & Control

17. Keep sensitive data in your own database

18. Control what information the model can access

RAG vs. Alternatives

RAG vs. Fine-Tuning

Aspect	RAG	Fine-Tuning
Cost	Low (inference only)	High (requires training)
Updates	Instant (add to database)	Slow (retrain model)
Transparency	Can see retrieved docs	Black box
Use Case	Dynamic knowledge	Specific behavior/style

RAG vs. Prompt Stuffing

Aspect	RAG	Prompt Stuffing
Token Limit	Efficient (only relevant context)	Limited (entire context)
Scalability	Scales to millions of docs	Limited to few pages
Relevance	Retrieves only relevant info	Must include everything

Challenges and Limitations

While powerful, RAG has challenges:

1. **Retrieval Quality:** If retrieval fails, generation suffers
2. **Chunking Strategy:** How you split documents matters
3. **Embedding Quality:** Poor embeddings = poor retrieval
4. **Context Length:** LLMs have limited context windows
5. **Latency:** Additional retrieval step adds time
6. **Cost:** More complex than simple LLM calls

We'll address all of these in later lessons.

What You've Learned

- RAG enhances LLMs with external knowledge retrieval
- It solves problems like hallucinations and outdated information
- The three steps: Indexing, Retrieval, Generation
- Real-world applications across industries
- Advantages over alternatives like fine-tuning

Next Steps

In [Lesson 2: Understanding Embeddings](#), you'll learn: - What vector embeddings are - How they capture semantic meaning - Different embedding models - How similarity search works

Further Reading

- [Original RAG Paper \(Lewis et al., 2020\)](#)
- [LangChain RAG Documentation](#)
- [Pinecone's Guide to RAG](#)

Practice Exercise: Before moving on, think about a use case where RAG would be valuable for you or your organization. What knowledge base would you want to query?

[← Back to README](#) | [Next: Understanding Embeddings →](#)